

1. Написание init программы

Сначала необходимо написать файл, который будет отвечать за процесс init. Для этого создаём файл init.c и пишем в нём небольшую программу, которая выводит на экран сообщение «Hello!» и засыпает на 20 секунд.

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 void main() {
5     printf("Hello!\n");
6     sleep(20);
7 }
```

Рисунок 1 — содержимое файла init.c

Затем данный файл необходимо собрать.

```
wild4est | wild4est-laptop | ~/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/tmp $ vim init.c
wild4est | wild4est-laptop | ~/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/tmp $ arm-linux-gnueabi-gcc -static init.c
-o init
```

Рисунок 2 — сборка init.c

Теперь полученный бинарный файл нужно упаковать в архив формата .cpio.gz.

```
wild4est | wild4est-laptop | ~/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/tmp $ echo init | cpio -o -H newc | gzip > i
nitramfs.cpio.gz
829 blocks
wild4est | wild4est-laptop | ~/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/tmp $
```

Рисунок 3 — упаковка файла init в архив .cpio.gz

После полученный архив нужно положить в одну папку вместе с zImage и dtb файлом.

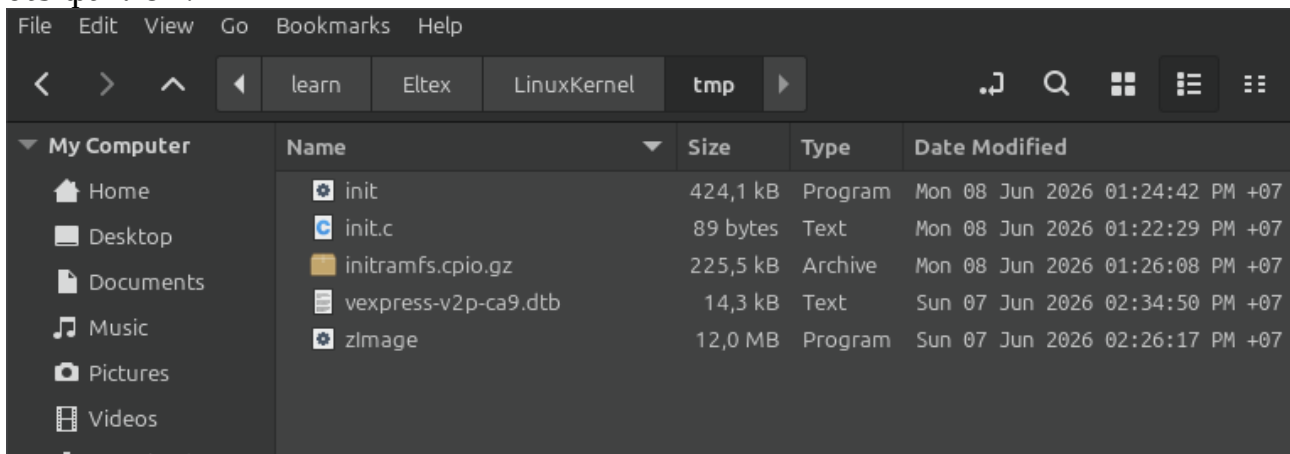


Рисунок 4 — содержимое директории tmp

Теперь из текущей папки можно запустить полученный образ ядра через qemu.

```
wild4est | wild4est-laptop | ~/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/tmp $ QEMU_AUDIO_DRV=none qemu-system-arm -M
vexpress-a9 -kernel zImage -initrd initramfs.cpio.gz -dtb vexpress-v2p-ca9.dtb -append "console=ttyAMA0" -nographic
[ 0.000000] Booting Linux on physical CPU 0x0
[ 0.000000] Linux version 7.0.11 (wild4est@wild4est-laptop) (arm-linux-gnueabi-hf-gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.
0, GNU ld (GNU Binutils for Ubuntu) 2.42) #1 SMP Sun Jun  7 14:26:02 +07 2026
[ 0.000000] CPU: ARMv7 Processor [410fc090] revision 0 (ARMv7), cr=10c5387d
[ 0.000000] CPU: DPT / VIRT / VIRT / VIRT / VIRT / VIRT / VIRT / VIRT / VIRT / VIRT / VIRT / VIRT / VIRT / VIRT / VIRT / VIRT
[ 2.468831] Run /init as init process
Hello!
[ 12.657112] amba 1000f000.wdt: deferred probe pending: (reason unknown)
[ 12.657368] amba 100e0000.memory-controller: deferred probe pending: (reason unknown)
[ 12.657427] amba 100e1000.memory-controller: deferred probe pending: (reason unknown)
[ 12.657460] amba 100e5000.watchdog: deferred probe pending: (reason unknown)
[ 22.560296] Kernel panic - not syncing: Attempted to kill init! exitcode=0x00000000
[ 22.565700] CPU: 0 UID: 0 PID: 1 Comm: init Not tainted 7.0.11 #1 VOLUNTARY
[ 22.567277] Hardware name: ARM-Versatile Express
[ 22.568651] Call trace:
[ 22.570409] unwind_backtrace from show_stack+0x10/0x14
[ 22.574670] show_stack from dump_stack_lvl+0x54/0x68
[ 22.575482] dump_stack_lvl from vpanic+0x234/0x454
[ 22.576249] vpanic from do_panic_on_target_cpu+0x0/0x14
[ 22.579619] ---[ end Kernel panic - not syncing: Attempted to kill init! exitcode=0x00000000 ]---
```

Рисунок 5-6 — результат запуска образа ядра.

По итогу после инициализации всех процессов можно увидеть, что ранее написанный процесс `init` выводит сообщение «Hello!» на экран, после чего ждёт 20 секунд и возникает ошибка `Kernel panic`, так как процесс `init` закончил свою работу.

2. Сборка и установка файловой системы при помощи BusyBox

Для начала необходимо сформировать дефолтный конфиг.

```
wild4est | wild4est-laptop | ~/BusyBox/busybox-1.36.1 $ make ARCH=arm defconfig
HOSTCC scripts/basic/fixdep
HOSTCC scripts/basic/split-include
HOSTCC scripts/basic/docproc
GEN include/applets.h
GEN include/usage.h
GEN networking/Kbuild
GEN networking/Config.in
```

Рисунок 7 — формирование дефолтного конфига

Теперь конфиг нужно подредактировать при помощи команды `make ACH=arm menuconfig`.

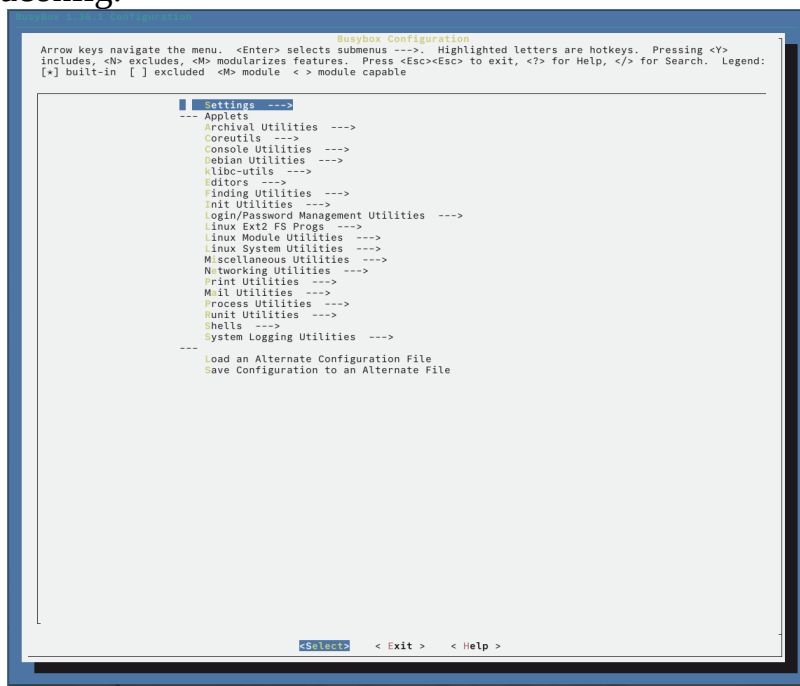


Рисунок 8 — окно конфигурации

В появившемся меню необходимо изменить несколько параметров, а именно добавить галочку рядом с «Build static binary» и добавить префикс кросс-компиляции.

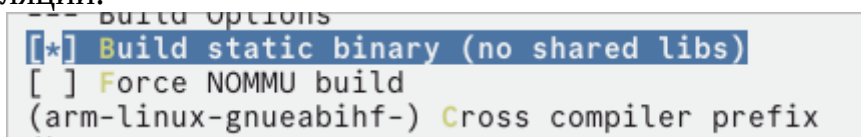


Рисунок 9 — параметры конфигурации

После можно собрать BusyBox при помощи команды `make`.

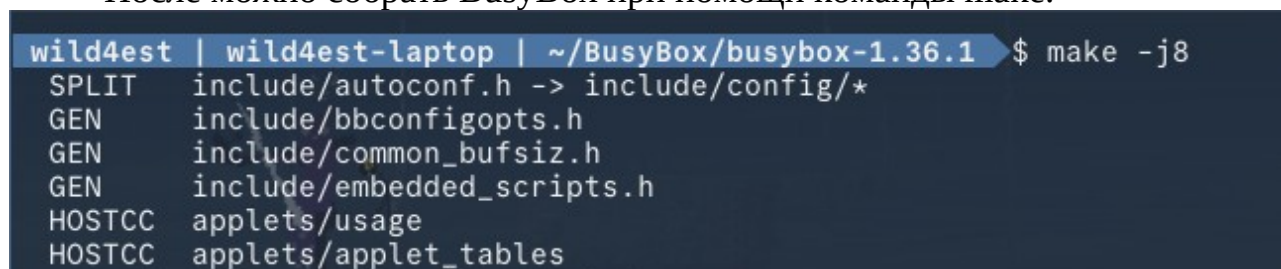


Рисунок 10 — сборка BusyBox

В процессе сборки в моём случае возникло ряд ошибок.

```
networking/tc.c: In function 'cbq_print_opt':
networking/tc.c:236:27: error: 'TCA_CBQ_MAX' undeclared (first use in this function); did you mean 'TCA_CBS_MAX'?
236 |         struct rtattr *tb[TCA_CBQ_MAX+1];
    |                             ^
    |                             TCA_CBS_MAX
networking/tc.c:236:27: note: each undeclared identifier is reported only once for each function it appears in
CC      miscutils/ubirename.o
CC      shell/match.o
networking/tc.c:249:16: error: 'TCA_CBQ_RATE' undeclared (first use in this function); did you mean 'TCA_TBF_RATE64'?
249 |         if (tb[TCA_CBQ_RATE]) {
    |                ^
    |                TCA_TBF_RATE64
CC      util-linux/volume_id/iso9660.o
CC      miscutils/volname.o
networking/tc.c:255:16: error: 'TCA_CBQ_LSSOPT' undeclared (first use in this function)
255 |         if (tb[TCA_CBQ_LSSOPT]) {
    |                ^
```

Рисунок 11 — ошибки, возникшие во время сборки

Данные ошибки возникли из-за того, что более свежие ядра Linux (начиная с версии 6.8) перестали поддерживать cbq queue, которые необходимы для сборки BusyBox.



Рисунок 12 — pull request проблемы, возникающей на ядрах Linux 6.8 и выше

Для того, чтобы избежать данных ошибок достаточно убрать параметр «tc» в разделе «Network Utilities».

```
[*] route (8.7 kb)
[*] slattach (6.2 kb)
[*] ssl_client (25 kb)
[ ] tc (8.3 kb)
[*] tcpsvd (14 kb)
[*] udpsvd (13 kb)
[*] telnet (8.8 kb)
```

Рисунок 13 — параметры конфигурации

Теперь после выполнения make ранее возникшие ошибки отсутствуют, следовательно можно сделать установку.

```
wild4est | wild4est-laptop | ~/BusyBox/busybox-1.36.1 $ make install
./_install//bin/arch -> busybox
./_install//bin/ash -> busybox
./_install//bin/base32 -> busybox
./_install//bin/base64 -> busybox
./_install//bin/cat -> busybox
./_install//bin/chatr -> busybox
./_install//bin/chgrp -> busybox
```

Рисунок 14 — установка BusyBox

После установки в директории BusyBox появилась папка «_install». Данную директорию необходимо зайти и все имеющиеся здесь файлы запаковать в архив .cpio.gz.

```
wild4est | wild4est-laptop | ~/BusyBox/busybox-1.36.1 $ cd _install/
wild4est | wild4est-laptop | ~/BusyBox/busybox-1.36.1/_install $ find . | cpio -o -H newc | gzip > initramfs.cpio.gz
cpio: File ./initramfs.cpio.gz grew, 786432 new bytes not copied
4645 blocks
```

Рисунок 15 — создание архива initramfs.cpio.gz

Теперь полученный файл можно скопировать в ту же директорию, где находятся zImage и файлы dtb.

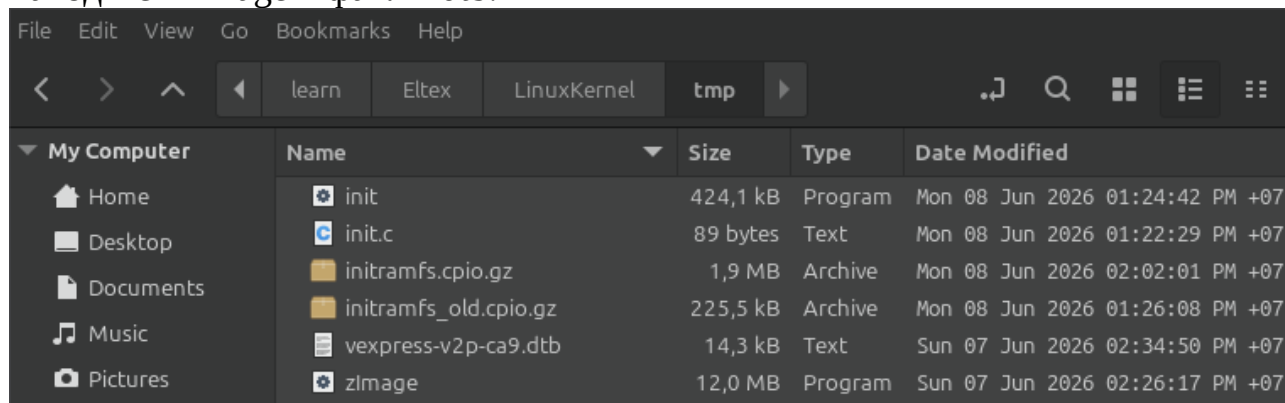


Рисунок 16 — содержимое директории tmp

Затем можно ещё раз запустить полученный образ ядра через qemu.

```
wild4est | wild4est-laptop | ~/Desktop/MyFavoriteFolder/learn/Eltex/LinuxKernel/tmp $ QEMU_AUDIO_DRV=none qemu-system-arm -M
vexpress-a9 -kernel zImage -initrd initramfs.cpio.gz -dtb vexpress-v2p-ca9.dtb -append "console=ttyAMA0 rdinit=/bin/ash" -nographic
[ 0.000000] Booting Linux on physical CPU 0x0
[ 0.000000] Linux version 7.0.11 (wild4est@wild4est-laptop) (arm-linux-gnueabi-hf-gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.
0, GNU ld (GNU Binutils for Ubuntu) 2.42) #1 SMP Sun Jun 7 14:26:02 +07 2026
[ 0.000000] CPU: ARMv7 Processor [410fc090] revision 0 (ARMv7), cr=10c5387d
[ 0.000000] CPU: PIPT / VIPT nonaliasing data cache, VIPT nonaliasing instruction cache
```

Рисунок 17 — запуск образа ядра.

В результате нам становится доступен командный интерпретатор.

```
[ 2.695295] Freeing unused kernel image (initramfs) memory: 2048K
[ 2.695295] Run /bin/ash as init process
/bin/ash: can't access tty; job control turned off
~ # [ 12.652831] amba 1000f000.wdt: deferred probe pending: (reason unknown)
[ 12.653167] amba 100e0000.memory-controller: deferred probe pending: (reason unknown)
[ 12.653230] amba 100e1000.memory-controller: deferred probe pending: (reason unknown)
[ 12.653272] amba 100e5000.watchdog: deferred probe pending: (reason unknown)
ls
bin      initramfs.cpio.gz  root      usr
dev      linuxrc            sbin
~ # echo "Hello!"
Hello!
~ #
```

Рисунок 18 — командный интерпретатор